

Introduction to Prompt Patterns

What are Prompt Patterns

Write a Python function that sorts a list of integers in ascending order.



Here is a simple Python function that sorts a list of integers in ascending order using the built-in `sorted()` function:

python

 코드 복사

```
def sort_list(numbers):  
    return sorted(numbers)  
  
# Example usage  
numbers = [5, 2, 9, 1, 5, 6]  
sorted_numbers = sort_list(numbers)  
print(sorted_numbers) # Output: [1, 2, 5, 5, 6, 9]
```

What are Prompt Patterns



"Write a Python function that sorts a list of integers in ascending order, following these steps:

1. Name the function `sort_numbers`.
2. Include a docstring that explains what the function does.
3. Use the built-in `sorted()` function.
4. Add an example usage in the main block to demonstrate how the function works."

What are Prompt Patterns



Here's a Python function that meets all the specified requirements:

python

코드 복사

```
def sort_numbers(numbers):  
    """  
    Sorts a list of integers in ascending order.  
  
    Parameters:  
        numbers (list): A list of integers to be sorted.  
  
    Returns:  
        list: A new list of integers sorted in ascending order.  
    """  
    return sorted(numbers)  
  
# Example usage  
if __name__ == "__main__":  
    numbers = [8, 3, 1, 7, 4, 10, 2]  
    sorted_numbers = sort_numbers(numbers)  
    print("Original list:", numbers)  
    print("Sorted list:", sorted_numbers)
```



Prompt Patterns that we will use

- Alternate Approach
- Cognitive Verifier
- Reflection

Alternate Approach

- Find multiple different approaches to solve the problem, and give us the best one

Alternate Approach



Write a function to iterate over each element in the list and add it to the sum if it is even.



Here's a Python function that iterates over each element in a list and adds it to a sum if it is even:

python

 코드 복사

```
def sum_of_evens(lst):  
    total = 0  
    for num in lst:  
        if num % 2 == 0: # Check if the number is even  
            total += num  
    return total
```

Alternate Approach



###Instruction###

When I ask a question, I need you to give me three answers. Additionally, provide pros and cons for each answer. I also want you to choose the best answer from the three answers you gave. Once I evaluate your answer and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###Reference###

Alternate Approach

Answer 1: Using a `for` loop

python

 코드 복사

```
def sum_even_numbers(lst):  
    total = 0  
    for num in lst:  
        if num % 2 == 0:  
            total += num  
    return total
```

- **Pros:**
 - Easy to understand and implement.
 - Suitable for beginners who are learning basic loops and conditionals.
- **Cons:**
 - Less concise compared to more advanced approaches.
 - Slightly slower than other methods for very large lists due to explicit looping.

Alternate Approach

Answer 2: Using list comprehension

python

 코드 복사

```
def sum_even_numbers(lst):  
    return sum([num for num in lst if num % 2 == 0])
```

- **Pros:**
 - More concise and compact.
 - Offers better readability for those familiar with list comprehensions.
- **Cons:**
 - Uses additional memory for creating the list comprehension.
 - May be less readable for beginners who are unfamiliar with this syntax.

Alternate Approach

Answer 3: Using the `filter` function

python

 코드 복사

```
def sum_even_numbers(lst):  
    return sum(filter(lambda x: x % 2 == 0, lst))
```

- **Pros:**
 - Efficient in terms of memory usage since it avoids creating a separate list.
 - Functional programming style can be more expressive for some users.
- **Cons:**
 - May be harder to understand for those not familiar with functional programming concepts like `filter` and `lambda`.
 - Slightly less readable compared to list comprehension.

Alternate Approach

Best Answer

The **list comprehension approach** is the best among the three because it strikes a balance between readability and performance. It is concise, and while it uses some additional memory, the impact is negligible for most practical use cases. It also provides better readability once one is familiar with the syntax.

Let me know if you would like me to refine the answer further!

Cognitive Verifier

- LLM asks multiple questions to make your prompt more specific

Cognitive Verifier

###Instruction###

When I ask a question, generate three additional questions that would help you provide a more accurate answer. Once I answer those questions, combine the answers to give me a final response to my original question.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###reference###

Cognitive Verifier



To better understand your requirements, here are three additional questions:

1. Should the function return the sum of all even numbers or perform some other operation with the sum, like printing it?
2. How would you like the function to handle non-integer elements in the list (if they exist)?
3. Should the function be written in a specific programming language, or is Python acceptable?

Let me know your answers, and I'll provide the final solution based on them!

1. Return only the sum.
2. Return Value Error
3. Python is acceptable

Cognitive Verifier



Based on your answers, here's a Python function that iterates over each element in a list, adds it to the sum if it's even, and returns the sum. If the list contains any non-integer elements, it will raise a `ValueError`.

python

코드 복사

```
def sum_of_evens(lst):  
    total = 0  
    for element in lst:  
        if not isinstance(element, int):  
            raise ValueError("List contains a non-integer element.")  
        if element % 2 == 0:  
            total += element  
    return total
```

This function will sum all even numbers in the list and return the result while ensuring that all elements are integers. If a non-integer is encountered, it raises a `ValueError`.

Reflection

- Let LLM give you reasons behind why it gave you such answer to your prompt
- Also give you considerations that you might want to take in improving the answer

Reflection

###Instruction###

When I ask a question, explain the reasoning behind your answer by breaking down the steps you took to arrive at it. Additionally, provide three alternative perspectives or considerations that could affect the accuracy or relevance of your answer. Once I evaluate those perspectives and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###Reference###

Reflection

python

 코드 복사

```
def sum_of_evens(numbers):  
    total = 0 # Initialize the sum to zero  
    for num in numbers: # Iterate over each element in the list  
        if num % 2 == 0: # Check if the element is even  
            total += num # Add it to the total if it is even  
    return total # Return the final sum
```

Reasoning Behind the Answer

1. **Initialization:** The function starts by initializing a variable `total` to zero, which will hold the sum of even numbers.
2. **Iteration:** It then iterates over each element in the list `numbers`.
3. **Condition Check:** For each element, it checks whether the number is even using the modulo operator (`%`). If the number is even (`num % 2 == 0`), it proceeds to the next step.
4. **Accumulation:** The even number is added to `total`.
5. **Return Statement:** After iterating through the list, the function returns the accumulated sum of all even numbers.



Reflection

Alternative Perspectives or Considerations

1. Using List Comprehension:

- The same functionality could be achieved with a more concise syntax using list comprehension combined with the `sum()` function:

python

 코드 복사

```
def sum_of_evens(numbers):  
    return sum(num for num in numbers if num % 2 == 0)
```

- This approach leverages Python's built-in functions for a more readable and concise implementation.

Reflection

2. Handling Non-Integer Values in the List:

- If the input list may contain non-integer values (e.g., floats, strings), the function should include additional checks to ensure it only processes integers:

python

 코드 복사

```
def sum_of_evens(numbers):  
    total = 0  
    for num in numbers:  
        if isinstance(num, int) and num % 2 == 0:  
            total += num  
    return total
```

- This version ensures type safety by checking if `num` is an integer.

Reflection

3. Using Functional Programming Approach:

- The `filter()` function can be combined with `sum()` to create a more functional style solution:

python

 코드 복사

```
def sum_of_evens(numbers):  
    return sum(filter(lambda x: x % 2 == 0, numbers))
```

- This approach might be preferable for those who are familiar with functional programming paradigms.

Please evaluate these perspectives and let me know which considerations to prioritize for a more refined solution.

Reflection

Please give me the answer while putting all three considerations into your solution.



Here's a solution that incorporates all three considerations into a single function. This version allows for more flexibility by handling different types of elements in the list, offering concise syntax, and adopting a functional programming approach where suitable.

python

 코드 복사

```
def sum_of_evens(numbers):  
    # Use a list comprehension to filter even integers and sum them up  
    return sum(  
        num for num in numbers  
        if isinstance(num, int) and num % 2 == 0 # Check if the element is an integer and  
    )
```

Reflection

Explanation

1. List Comprehension for Conciseness:

- The solution uses a list comprehension to filter even integers from the input list, making the code concise and readable.
- The expression `num for num in numbers if ...` generates the numbers that satisfy the conditions for inclusion in the sum.

2. Handling Non-Integer Values:

- The check `isinstance(num, int)` ensures that only integer elements are considered, preventing errors if non-integer values are present in the list.
- This guarantees that the function won't attempt to compute the modulo operation on incompatible types like strings or floats.

Reflection

3. Functional Style for Clarity:

- Although the list comprehension itself is already compact, this solution aligns with a functional programming style by directly summing the filtered elements in a single expression.
- Using this approach, we avoid explicitly initializing and updating a variable (`total`), which can make the code easier to understand.

This function combines readability, type safety, and functional programming techniques to provide a well-rounded solution. Let me know if any additional refinements or considerations are needed!

Prompt Pattern Form – Alternate Approach

###Instruction###

When I ask a question, I need you to give me three answers. Additionally, provide pros and cons for each answer. I also want you to choose the best answer from the three answers you gave. Once I evaluate your answer and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

(Fill in here!)

###Reference###

(Fill in here if needed)

Prompt Pattern Form – Cognitive Verifier

###Instruction###

When I ask a question, generate three additional questions that would help you provide a more accurate answer. Once I answer those questions, combine the answers to give me a final response to my original question.

###Question###

(Fill in here!)

###reference###

(Fill in here if needed)

Prompt Pattern Form – Reflection

###Instruction###

When I ask a question, explain the reasoning behind your answer by breaking down the steps you took to arrive at it. Additionally, provide three alternative perspectives or considerations that could affect the accuracy or relevance of your answer. Once I evaluate those perspectives and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

(Fill in here!)

###Reference###

(Fill in here if needed)